



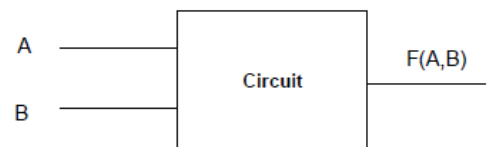
Module	Structure Machine	
Filière	MI	1 ^{ère} Année S1

Algèbre de Boole.

Définitions & Axiomes.

Les machines numériques sont constituées d'un ensemble de **circuits** électroniques. Chaque circuit fournit une **fonction logique** bien déterminée (addition, comparaison...etc.).

- La fonction logique $F(A, B)$ peut être : la somme de A et B , ou le résultat de la comparaison de A et B ou une autre fonction.



Pour **concevoir et réaliser** un circuit on doit avoir le modèle **mathématique de la fonction** réalisée par ce dernier, tel que ce modèle doit prendre en considération le **système binaire**. Le modèle mathématique utilisé est celui de **Boole** (George Boole est un mathématicien anglais :1815-1864).

George Boole mathématicien autodidacte anglais, a développé une algèbre permettant de manipuler les propositions logiques au moyen d'équations mathématiques où les énoncés VRAI et FAUX sont représentés par les valeurs 1 et 0, tandis que les opérateurs ET et OU deviennent des opérateurs algébriques de multiplication et d'addition. Le présent chapitre est consacré à cette algèbre, présentant dans un premier temps les postulats, les axiomes et les théorèmes qui en découlent. Une partie du chapitre est également consacrée à la manipulation de cette algèbre logique, suivie d'une illustration des applications possibles pour les besoins des circuits logiques. Pour ce faire, une introduction des fondements portes logiques est présentée.

Ainsi, l'algèbre de Boole est un très bon outil utilisant des règles dont les quels les **fonctions** (expressions) sont constitués par des **variables** qui peuvent prendre les valeurs '**OUI**' ou '**NON**' (sont **bien adaptés** au Système **binaire** (0 et 1)). Elle est utilisée pour faire l'étude des systèmes qui possèdent **deux états s'excluent mutuellement** :

- Le système peut être uniquement dans **deux états** $E1$ et $E2$ tel que $E1$ est l'opposé de $E2$.
- Le système **ne peut pas être** dans l'état $E1$ et $E2$ en **même temps**.

1. DEFINITIONS ET CONVENTIONS

1.1. NIVEAU LOGIQUE

Lorsque on fait l'étude d'un système logique il faut bien préciser le niveau du travail.

Niveau	Logique positive	Logique négative
H (High) haut	1	0
L (Low) bas	0	1

Exemple :

Logique positive :

Lampe allumée : 1
Lampe éteinte : 0

Logique négative

Lampe allumée : 0
Lampe éteinte : 1

1.2. VARIABLE LOGIQUE (BOOLEENNE)

Une variable logique (booléenne) est une variable qui peut prendre soit la valeur 0 ou 1. Généralement elle est exprimée par un seul caractère alphabétique en majuscule (A, B, S, ...).

Exemple :

- Une lampe : allumée $L = 1$, éteinte $L = 0$.
- 1^{er} interrupteur ouvert : $I_1 = 1$ fermé : $I_1 = 0$,
- 2^{ème} interrupteur ouvert : $I_2 = 1$

1.3. OPERATEURS LOGIQUES DE L'ALGÈBRE DE BOOLE

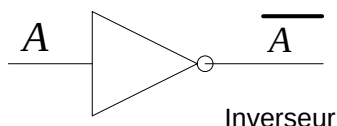
On appelle B l'ensemble constitué de deux éléments appelés **valeurs de vérité** {VRAI, FAUX}. Cet ensemble est aussi noté $B = \{1, 0\}$.

Sur cet ensemble on peut définir deux lois (opérations ou fonctions) : ET et OU et une transformation appelée NON, complémentaire, inversion ou contraire. Ces opérations on les appelle les opérations logiques de base.

NON : (Négation) : est un opérateur unaire (une seule variable) qui a pour rôle d'**inverser** la valeur d'une variable. Tel que Le contraire d'une variable "A" est VRAI si et seulement si A est FAUX. Le contraire de A est noté $\bar{A}$. (lire : A barre)

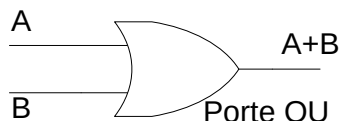
A	$\bar{A}$
0	1
1	0

Table de vérité



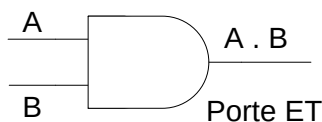
Le **OU (OR)** est un opérateur binaire (deux variables), a pour rôle de réaliser la **somme logique** entre deux variables logiques. Ainsi, il fait la **disjonction** entre deux variables. Le OU est défini par : $F(A, B) = A + B$ (il ne faut pas confondre avec la somme arithmétique).

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



Le **ET (AND)** est un opérateur binaire (deux variables), a pour rôle de réaliser le **Produit logique** entre deux variables booléennes. L'opérateur **ET** fait la **conjonction** entre deux variables. Il est défini par $F(A, B) = A \cdot B$ (il ne faut pas confondre avec la multiplication arithmétique).

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

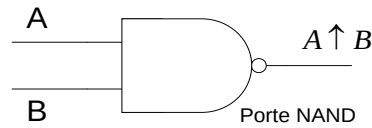
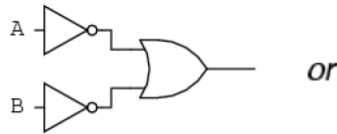


L'opérateur **NON-ET** (NAND en anglais) associe un résultat qui a lui-même la valeur VRAI seulement si au moins l'un des deux opérandes a la valeur FAUX. Il est défini comme suit :

$$F(A, B) = \overline{A \cdot B}$$

$$F(A, B) = A \uparrow B$$

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

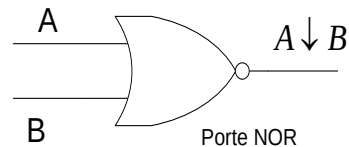
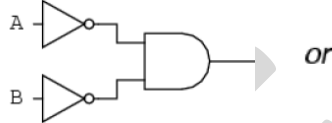


L'opérateur **NON-OU** (NOR en anglais) associe un résultat qui a lui-même la valeur VRAI seulement si les deux opérandes ont la valeur FAUX. Il est défini comme suit :

$$F(A, B) = \overline{A + B}$$

$$F(A, B) = A \downarrow B$$

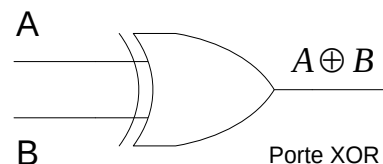
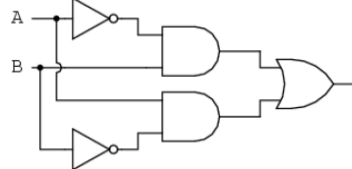
A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



L'opérateur **OU exclusif**, souvent appelée **XOR** (eXclusive **OR**), associe un résultat qui a lui-même la valeur VRAI seulement si les deux opérandes ont des valeurs distinctes. Il est défini comme suit :

$$F(A, B) = A \oplus B = \overline{A}B + A\overline{B}$$

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



Remarque :

- Dans la définition des opérateurs ET, OU, nous avons juste donner la définition de base avec deux variables logiques.
- On peut noter A non XOR B par $A \otimes B$ et on lit A XNOR B.
- L'opérateur ET peut réaliser le produit de plusieurs variables logique (ex : $A \cdot B \cdot C \cdot D$).
- L'opérateur OU peut aussi réaliser la somme logique de plusieurs variables logiques (ex : $A + B + C + D$).
- Dans une expression on peut aussi utiliser les parenthèses.
- Les portes ET, OU, NAND, NOR peuvent avoir plus que deux entrées
- Il n'existe pas de OU exclusif à plus de deux entrées

1.4. LOIS FONDAMENTALES DE L'ALGÈBRE DE BOOLE

PROPRIETES DE L'OPERATEUR « OU »

- ✓ $a + 1 = 1$ élément absorbant
- ✓ $a + 0 = a$ élément neutre
- ✓ $a + a = a$ idempotence
- ✓ $a + \overline{a} = 1$ complémentarité
- ✓ $a + b = b + a$ commutativité
- ✓ $a + b + c = (a + b) + c = a + (b + c)$ associativité

PROPRIETES DE L'OPERATEUR « ET »

- ✓ $a \cdot 1 = a$ élément neutre
- ✓ $a \cdot 0 = 0$ élément absorbant
- ✓ $a \cdot a = a$ idempotence
- ✓ $a \cdot \overline{a} = 0$ complémentarité
- ✓ $a \cdot b = b \cdot a$ commutativité
- ✓ $a \cdot b \cdot c = (a \cdot b) \cdot c = a \cdot (b \cdot c)$ associativité

PROPRIÉTÉS DE L'OPÉRATEUR « NON »	PROPRIÉTÉ DE LA DISTRIBUTIVITÉ
$\overline{\overline{A}} = A$	✓ $a \cdot (b + c) = a \cdot b + a \cdot c$
$\overline{A + A} = 1$	✓ $(a + b) \cdot (c + d) = a \cdot c + a \cdot d + b \cdot c + b \cdot d$
$\overline{A \cdot A} = 0$	✓ $a + (b \cdot c) = (a + b) \cdot (a + c)$
PROPRIÉTÉS DES OPÉRATEURS « NAND »	PROPRIÉTÉS DES OPÉRATEURS « NOR »
$A \uparrow 0 = 1$	$A \downarrow 0 = \overline{A}$
$A \uparrow 1 = \overline{A}$	$A \downarrow 1 = 0$
$A \uparrow B = B \uparrow A$	$A \downarrow B = B \downarrow A$
$(A \uparrow B) \uparrow C \neq A \uparrow (B \uparrow C)$	$(A \downarrow B) \downarrow C \neq A \downarrow (B \downarrow C)$

Remarque 1 :

Toutes ces formules permettent déjà de simplifier des fonctions logiques, c'est à dire d'éliminer au maximum les opérateurs logiques (sans bien sur modifier la fonction initiale !)

Exemple : Soit S à simplifier $S = (X + \overline{Y}) \cdot (X + Y) + Z \cdot (\overline{X} + Y)$

- Par distributivité $S = (X + \overline{Y}) \cdot X + (X + \overline{Y}) \cdot Y + Z \cdot (\overline{X} + Y)$
- Par distributivité $S = X \cdot X + \overline{Y} \cdot X + X \cdot Y + \overline{Y} \cdot Y + Z \cdot \overline{X} + Z \cdot Y$
- Par idempotence ($X \cdot X = X$) $S = X + \overline{Y} \cdot X + X \cdot Y + \overline{Y} \cdot Y + Z \cdot \overline{X} + Z \cdot Y$
- Par complémentarité $Y \cdot \overline{Y} = 0$ $S = X + \overline{Y} \cdot X + X \cdot Y + Z \cdot \overline{X} + Z \cdot Y$
- Par identité remarquable ($1 \cdot X = X$) $S = 1 \cdot X + \overline{Y} \cdot X + X \cdot Y + Z \cdot \overline{X} + Z \cdot Y$
- Par distributivité $S = X \cdot (1 + \overline{Y} + Y) + Z \cdot \overline{X} + Z \cdot Y$
- Par identité remarquable (sur + puis .) $S = X + Z \cdot \overline{X} + Z \cdot Y$

Nous pourrions même simplifier encore la fonction S quand nous aurons vu quelques autres formules un peu plus "puissantes" que celles vues ci-dessus.

Remarque 2:

NAND et NOR sont des opérateurs **universels** (complets) c.-à-d. en les utilisant, on peut exprimer n'importe quelle expression (fonction) logique, et Pour cela, Il suffit d'exprimer les opérateurs de base (NON, ET, OU) avec des NAND et des NOR.

Exemple : Réalisation des opérateurs de base avec des NOR

$$\overline{A} = \overline{A + A} = A \downarrow A$$

$$A + B = \overline{\overline{A + B}} = \overline{A \downarrow B} = (A \downarrow B) \downarrow (A \downarrow B)$$

$$A \cdot B = \overline{\overline{A \cdot B}} = \overline{A \downarrow B} = (A \downarrow A) \downarrow (B \downarrow B)$$

1.5. DUALITE DE L'ALGÈBRE DE BOOLE

Toute expression logique reste **vraie** si on remplace le ET par le OU, le OU par le ET, le 1 par 0, le 0 par 1.

Exemple :

$$A + 1 = 1 \Leftrightarrow A \cdot 0 = 0$$

$$A + \overline{A} = 1 \Leftrightarrow A \cdot \overline{A} = 0$$

1.6. THEOREME DE DE-MORGANE

- La somme logique complémentée de deux variables est égale au produit des compléments des deux variables.

$$\overline{A+B} = \bar{A} \cdot \bar{B}$$

- Le produit logique complémenté de deux variables est égal au somme logique des compléments des deux variables.

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

1.6.1. GENERALISATION DU THEOREME DE-MORGANE A N VARIABLES

$$\overline{A \cdot B \cdot C \dots} = \bar{A} + \bar{B} + \bar{C} + \dots$$

$$\overline{A + B + C + \dots} = \bar{A} \cdot \bar{B} \cdot \bar{C} \dots$$

1.7. DECOMPOSITION DE SHANNON

Lorsque Shannon introduisit l'algèbre en vue d'une utilisation dans le cadre des circuits à relais, il ajouta une notion importante, connue aujourd'hui sous le nom de la décomposition de Shannon.

La décomposition de Shannon est très utile pour la simplification des *fonctions logiques*¹, et elle nous servira pour mieux comprendre le fonctionnement de certains circuits usuels vus plus loin dans ce cours. De plus, elle permet de poser des démonstrations élégantes pour des théorèmes récalcitrants. La décomposition de Shannon s'énonce comme suit :

Soit f une fonction logique de paramètres $x_1, x_2, \dots, x_n$, alors :

$$f(x_1, x_2, \dots, x_n) = \bar{x}_1 f(0, x_2, \dots, x_n) + x_1 f(1, x_2, \dots, x_n)$$

Preuve

- Si $x_1 = 0$, le côté gauche de l'égalité vaut $f(0, x_2, \dots, x_n)$, alors que le côté droit vaut : $1.f(0, x_2, \dots, x_n) + 0.f(1, x_2, \dots, x_n) = f(0, x_2, \dots, x_n)$.

- Si $x_1 = 1$, le côté gauche de l'égalité vaut $f(1, x_2, \dots, x_n)$, alors que le côté droit vaut : $0.f(0, x_2, \dots, x_n) + 1.f(1, x_2, \dots, x_n) = f(1, x_2, \dots, x_n)$

Dans les deux cas, les deux côtés de l'égalité prennent la même valeur. Aussi l'égalité est prouvée. Notons ici que la variable « x_1 » peut être remplacée par n'importe quelle autre variable parmi les $n-1$ variables restantes.

La décomposition peut également être appliquée récursivement sur l'ensemble des variables. De plus, en appliquant le principe de dualité, on peut trouver que, pour toute fonction logique f de paramètres $x_1, x_2, \dots, x_n$, il est possible d'écrire :

$$f(x_1, x_2, \dots, x_n) = (\bar{x}_1 + f(1, x_2, \dots, x_n))(x_1 + f(0, x_2, \dots, x_n))$$

Exemple

Soit $f(a, b) = \overline{a+b}$, suivant la décomposition de Shannon, on note :

$$f(a, b) = \bar{a} \cdot f(0, b) + a \cdot f(1, b) \text{ Or,}$$

$$f(0, b) = \overline{0+b} = \bar{b} \text{ et } f(1, b) = \overline{1+b} = \bar{1} = 0$$

Il en ressort que

$$f(a, b) = \bar{a} \bar{b} + a \cdot 0 = \bar{a} \bar{b}; \text{ CQFD}$$

¹ Nous appelons fonction logique ce que nous définissions précédemment comme une expression logique. Les paramètres de la fonction sont simplement les variables que cette expression fait intervenir.

2. FONCTION LOGIQUE

C'est une fonction qui relie N variables logiques avec un ensemble d'opérateurs logiques de base (NON, ET, OU). Sa valeur est égale à 1 ou 0 selon les valeurs des variables logiques.

- Si une fonction logique possède N variables logiques $\Rightarrow 2^n$ combinaisons $\Rightarrow$ la fonction possède 2^n valeurs.
- Les 2^n combinaisons sont représentées dans une table qui s'appelle table de vérité (TV).

Exemple :

$$F(A, B, C) = \bar{A}\bar{B}.C + \bar{A}.B.C + A.\bar{B}.C + A.B.C$$

La fonction possède 3 variables $\rightarrow 2^3$ combinaisons

$$F(0,0,0) = \bar{0}\bar{0}.0 + \bar{0}.0.0 + 0.\bar{0}.0 + 0.0.0 = 0$$

$$F(0,0,1) = \bar{0}\bar{0}.1 + \bar{0}.0.1 + 0.\bar{0}.1 + 0.0.1 = 1$$

$$F(0,1,0) = \bar{0}\bar{1}.0 + \bar{0}.1.0 + 0.\bar{1}.0 + 0.1.0 = 0$$

$$F(0,1,1) = \bar{0}\bar{1}.1 + \bar{0}.1.1 + 0.\bar{1}.1 + 0.1.1 = 1$$

$$F(1,0,0) = \bar{1}\bar{0}.0 + \bar{1}.0.0 + 1.\bar{0}.0 + 1.0.0 = 0$$

$$F(1,0,1) = \bar{1}\bar{0}.1 + \bar{1}.0.1 + 1.\bar{0}.1 + 1.0.1 = 1$$

$$F(1,1,0) = \bar{1}\bar{1}.0 + \bar{1}.1.0 + 1.\bar{1}.0 + 1.1.0 = 0$$

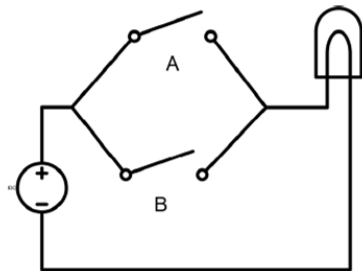
$$F(1,1,1) = \bar{1}\bar{1}.1 + \bar{1}.1.1 + 1.\bar{1}.1 + 1.1.1 = 1$$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

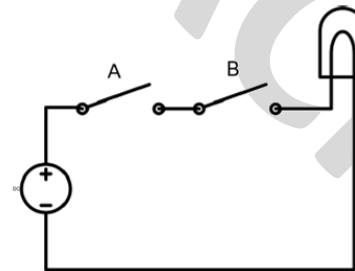
Une table de vérité

2.1. NOTION D'UN CIRCUIT LOGIQUE (LOGIGRAMME)

Le lien établi entre l'algèbre de Boole et les circuits logiques date du début du XXe siècle. Ce fut une véritable révolution dont nous connaissons largement les conséquences aujourd'hui. Il s'agissait initialement d'une application aux circuits à relais (sorte d'interrupteurs).

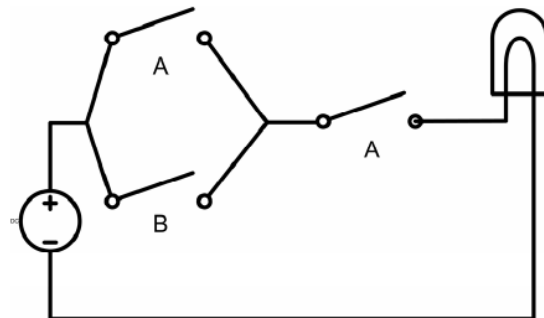


La lampe s'allume si A OU B est fermé



La lampe s'allume si A ET B sont fermés

Si des relais obéissaient à la même commande (variable), alors une fonction logique pouvait exprimer son fonctionnement général :



La lampe s'allume si (A OU B) ET A est fermé;

Donc la lampe s'allume si A est fermé puisque

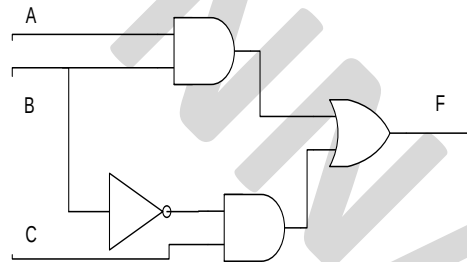
$$A(A+B) = A$$

Un circuit logique est la traduction de la **fonction logique** en un schéma électronique, tel que le principe consiste à remplacer chaque opérateur logique par la porte logique qui lui correspond. Les **portes logiques** sont les éléments de base à l'aide desquels nous pouvons exprimer toute fonction logique. Une question légitime qui se pose alors est : « Pourquoi disposons-nous d'autant de portes logiques, puisque toute fonction logique peut être exprimée à l'aide des opérateurs ET, OU et NON ? »

Il y a **deux raisons** majeures à cela. La **première** est d'ordre technologique, tenant compte du fait que la réalisation d'une porte ET à l'aide de transistors présuppose l'existence d'une porte NON-ET, qui à son tour est inversée. La réciproque est tout aussi vraie pour la porte OU et la porte NON-OU. De la même façon, les portes XOR et XNOR sont très utiles et leur réalisation matérielle est plus simple que l'équivalent utilisant des portes ET, OU et NON. Cette réalité va nous amener à la notion de **coût d'un circuit**. La **seconde** raison tient compte du fait que toute fonction logique peut être exprimée au moyen de NON-ET uniquement (respectivement, à l'aide de NON-OU uniquement) car ils sont des opérateurs complets.

Le passage d'une fonction logique à un circuit logique induit certaines modifications aux notions vues précédemment. Les variables d'une fonction logique deviennent les entrées du circuit, ce circuit donnant en sortie la valeur de la fonction logique suivant la valeur des entrées. On peut ainsi connecter des portes logiques les unes aux autres pour réaliser une fonction logique. À l'inverse, trouver la fonction logique réalisée par un circuit nous permettra de le manipuler en vue de simplifications éventuelles.

Exemple :



$$F(A, B, C) = AB + \bar{B}.C$$

2.2. COUT D'UN CIRCUIT

Afin de décider quelle implémentation d'une fonction logique est la plus appropriée, on propose une **métrique d'évaluation** du coût d'un circuit. Cette métrique n'est pas à proprement parler exacte et universelle, mais elle va nous permettre de mettre une valeur numérique sur un circuit et de décider de son caractère économique.

La métrique s'énonce comme suit :

- ✓ Une porte (ET/OU) **inversée** avec N entrées à un coût de **N+1**.
- ✓ Une porte (ET/OU) **non inversée** avec N entrées à un coût de **N+2**.

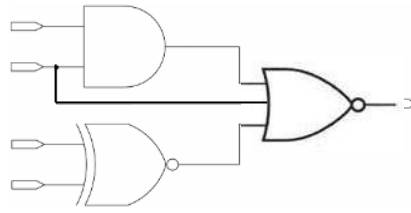
On fait une distinction entre les portes inversées et non inversées (on veut dire dont la sortie est inversée) pour refléter la réalité de la technologie CMOS (**Complementary Metal Oxide Semiconductor, une technologie de fabrication de composants électronique**) dans laquelle une porte non inversée (ET, OU) est réalisée en faisant suivre d'un inverseur la version inversée de cette porte (NON-ET, NON-OU) :



- ✓ Pour des raisons de simplification des calculs on ne considère pas le coût des inverseurs à l'entrée d'une porte.
- ✓ Il arrive néanmoins que nous voulions évaluer le coût d'un circuit comportant des portes XOR ou XNOR. Par convention, on admettra que le coût associé à ces portes est de **2N**, N étant le nombre d'entrée.

Exemple :

Pour le circuit suivant :



Nous avons :

- Une porte logique ET à deux entrées : $2+2=4$.
- Une porte logique XNOR à deux entrées : $2*2=4$.
- Une porte logique NOR à trois entrées : $3+1=4$.
- **Coût total = $4+4+4 = 12$**

3. DEFINITION TEXTUELLE D'UNE FONCTION LOGIQUE

Généralement la définition du fonctionnement d'un système est donnée sous un **format textuel**. Ainsi, pour faire **l'étude et la réalisation** d'un tel système on doit avoir son **modèle mathématique** (fonction logique) $\Rightarrow$ il faut **tirer (déduire)** la **fonction logique** à partir de la **description textuelle**.

Exemple :

Une serrure de sécurité s'ouvre en fonction de **trois clés**. Le fonctionnement de la serrure est défini comme suite :

- La serrure est ouverte si au moins deux clés sont utilisées.
- La serrure reste fermée dans les autres cas.

Donner le schéma du circuit qui permet de contrôler l'ouverture de la serrure ?

- Le système possède **trois entrées** : chaque entrée représente une clé.
- On va correspondre à chaque clé une variable logique : clé₁ $\Rightarrow$ A, la clé₂ $\Rightarrow$ B, la clé₃ $\Rightarrow$ C
 - Si la clé₁ est utilisée alors la variable $A=1$ sinon $A=0$
 - Si la clé₂ est utilisée alors la variable $B=1$ sinon $B=0$
 - Si la clé₃ est utilisée alors la variable $C=1$ sinon $C=0$
- Le système possède **une seule sortie** qui correspond à l'état de la serrure (ouverte ou fermée).
- On va correspondre une variable S pour designer la sortie :
 - $S=1$ si la serrure est ouverte,
 - $S=0$ si elle est fermée

$$S=F(A,B,C)=\begin{cases} F(A,B,C)=1 \text{ si au moins deux clés sont introduites} \\ F(A,B,C)=0 \text{ sinon.} \end{cases}$$

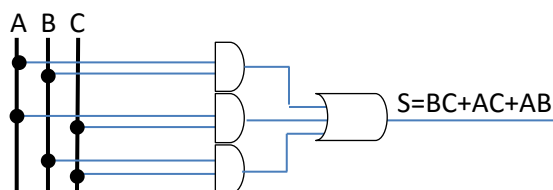
Ainsi, la table de vérité du système sera comme suit :

A	B	C	S
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

D'après la table de vérité on a :

- $S=F(A, B, C)=\bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$
- $S=BC+AC+AB$

Alors le circuit correspondant est le suivant :



4. FORME CANONIQUE D'UNE FONCTION LOGIQUE

On appelle **forme canonique** d'une fonction la forme où chaque **terme** de la fonction comporte **toutes les variables**. On l'appelle canonique car elle est **unique** pour chaque fonction (Une expression canonique n'est cependant pas optimale).

On identifie, selon le principe de dualité propre à l'algèbre de Boole, deux formes canoniques : la **somme de produits** (appelé forme canonique **disjonctive** ou la somme des **minterms**). Une somme de produits est sous forme canonique si toutes les variables apparaissent dans tous les termes des produits qui la composent, et le **produit de sommes** (appelé forme canonique **conjonctive** ou produit de **maxterms**). Un produit de sommes est sous forme canonique si toutes les variables apparaissent dans tous les termes de sommes qui le composent.

Ligne	A	B	C	F	Minterms m_i	Maxterms M_i
0	0	0	0	0	$\bar{A}\bar{B}\bar{C}$	$A + B + C$
1	0	0	1	0	$\bar{A}\bar{B}C$	$A + B + \bar{C}$
2	0	1	0	0	$\bar{A}B\bar{C}$	$A + \bar{B} + C$
3	0	1	1	1	$\bar{A}BC$	$A + \bar{B} + \bar{C}$
4	1	0	0	0	$A\bar{B}\bar{C}$	$\bar{A} + B + C$
5	1	0	1	1	$A\bar{B}C$	$\bar{A} + B + \bar{C}$
6	1	1	0	1	$AB\bar{C}$	$\bar{A} + \bar{B} + C$
7	1	1	1	1	ABC	$\bar{A} + \bar{B} + \bar{C}$

On en déduit l'expression de F sous forme canonique disjonctive et ses écritures condensées :

$$\begin{aligned} \checkmark \quad F(A, B, C) &= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC \\ F(A, B, C) &= m_3 + m_5 + m_6 + m_7 \end{aligned}$$

On en déduit l'expression de F sous forme canonique conjonctive et ses écritures condensées :

$$\begin{aligned} \checkmark \quad F(A, B, C) &= \bar{F}(A, B, C) = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(\bar{A} + B + C) \\ F(A, B, C) &= M_0.M_1.M_2.M_4 \end{aligned}$$

Il existe une autre représentation des formes canoniques d'une fonction logique, cette représentation est appelée forme numérique.

- **R** ou $\sum$: pour indiquer la forme disjonctive.
- **P** ou $\prod$: pour indiquer la forme conjonctive.

Si on reprend la fonction de l'exemple précédent :

$$\begin{aligned} \checkmark \quad F(A, B, C) &= \sum(3, 5, 6, 7) = R(011, 101, 110, 111) = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC. \\ \checkmark \quad F(A, B, C) &= \prod(0, 1, 2, 4) = P(000, 001, 010, 100) = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)(\bar{A} + B + C) \end{aligned}$$

Remarque :

- On peut toujours **ramener n'importe** qu'elle fonction logique à l'une des **formes canoniques**.
- Cela revient à rajouter les variables manquantes dans les termes qui ne **contiennent** pas toutes les variables (les termes non canoniques), en utilisant les règles de l'algèbre de Boole :
 - Multiplier un terme avec une expression qui vaut 1.
 - Additionner à un terme avec une expression qui vaut 0.
 - Par la suite faire la distribution
- La **première et la deuxième** forme canonique sont **équivalentes**.
- Il est possible de passer d'une expression canonique disjonctive à une expression canonique conjonctive (et vice et versa) en considérant les termes manquants **dans la duale**.

5. SIMPLIFICATION DES FONCTIONS LOGIQUES

Pourquoi ?

- ✓ Utiliser le moins de composants possibles ;
- ✓ Simplifier au maximum le schéma de câblage en réduisant le nombre de portes logiques utilisées $\Rightarrow$ réduction du coût du circuit.

Il faut donc trouver la forme minimale de l'expression logique considérée, et pour cela on doit :

- ✓ Réduire le nombre de terme dans une fonction ;
- ✓ Réduire le nombre de variable dans un terme.

Trois méthodes

- ✓ Algébrique (en utilisant des propriétés et des théorèmes)
- ✓ Graphique (tableaux de Karnaugh; ...).
- ✓ Programmable (MÉTHODE DE QUINE-MAC CLUSKEY)

5.1. SIMPLIFICATION ALGÈBRE

Cette méthode n'a pas une démarche bien spécifique, son principe consiste à appliquer les règles de l'algèbre de Boole afin d'éliminer des variables ou des termes.

Ainsi, Cette technique de simplification repose sur l'utilisation des théorèmes fondamentaux et des propriétés de l'algèbre de Boole. Après la recherche de l'expression algébrique de la fonction, l'étape suivante consiste à minimiser le nombre de termes dans une fonction afin d'obtenir un circuit plus petit donc plus facile à construire avec un coût plus réduit.

La simplification algébrique repose sur beaucoup d'astuce, quand la fonction est plus complexe (au-delà de trois variables) cette méthode de simplification devient peu évidente. On peut:

- ✓ Supprimer les associations de termes multiples.
- ✓ Mettre en facteur des variables pour éliminer plusieurs termes.
- ✓ Mettre en facteur des variables pour faire apparaître des termes inclus.
- ✓ Ajouter un terme qui existe déjà à une expression logique.

Quelques règles fondamentales :

- ✓ **Règle 1 :** Dans une somme on peut éliminer tous les multiples d'un terme fondamental.

$$X + XY = X$$

- ✓ **Règle 2 :** (Absorption) : Dans la somme d'un terme et d'un multiple de son complément, on peut éliminer le complément. $X + \bar{X}Y = X + Y$. ($= X + XY + \bar{X}Y = X + Y(X + \bar{X}) = X + Y$).

- ✓ **Règle 3 :** Regrouper des termes à l'aide des règles de l'algèbre de Boole.

$$ABC + A\bar{B}C + A\bar{B}C\bar{D} = AB(C + \bar{C}) + A\bar{B}C\bar{D} = AB + A\bar{B}C\bar{D} = A(B + \bar{B}(C\bar{D})) = A(B + C\bar{D}) = AB + ACD$$

- ✓ **Règle 4 :** Rajouter un terme déjà existant à une expression

$$ABC + \bar{A}BC + A\bar{B}C + ABC = ABC + \bar{A}BC + ABC + A\bar{B}C + ABC + ABC = BC + AC + AB$$

- ✓ **Règle 5 :** Il est possible de supprimer un terme superflu (un terme en plus), c'est-à-dire déjà inclus dans la réunion des autres termes.

$$F(A, B, C) = AB + \bar{B}C + AC = AB + \bar{B}C + AC(B + \bar{B})$$

$$= AB + \bar{B}C + ACB + A\bar{B}C = AB(1 + C) + \bar{B}C(1 + A) = AB + \bar{B}C$$

- ✓ **Règle 6 :** Il est préférable de simplifier la forme canonique ayant le nombre de termes minimum

$$F(A, B, C) = R(2, 3, 4, 5, 6, 7)$$

$$\overline{F(A, B, C)} = R(0, 1) = \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot \bar{B} \cdot C$$

$$= \bar{A} \cdot \bar{B}(\bar{C} + C)$$

$$= \bar{A} \cdot \bar{B} = \overline{A + B}$$

$$F(A, B, C) = \overline{\overline{F(A, B, C)}} = \overline{\bar{A} \cdot \bar{B}} = A + B$$

5.2. SIMPLIFICATION PAR LES TABLEAUX DE KARNAUGH**5.2.1. LES TERMES ADJACENTS**

On dit que deux termes sont adjacents s'ils possèdent les mêmes variables et qui ne diffèrent que par une seule variable à la fois (une seule variable change l'état à la fois).

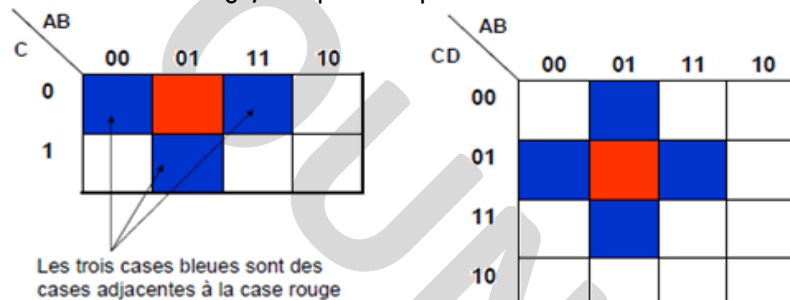
Examinons l'expression suivante : $A \cdot B + A \cdot \bar{B}$

- ✓ Les deux termes possèdent les mêmes variables. La seule différence est que l'état de la variable **B change**. Si on applique les règles de simplification on obtient :

$$AB + A\bar{B} = A(B + \bar{B}) = A$$

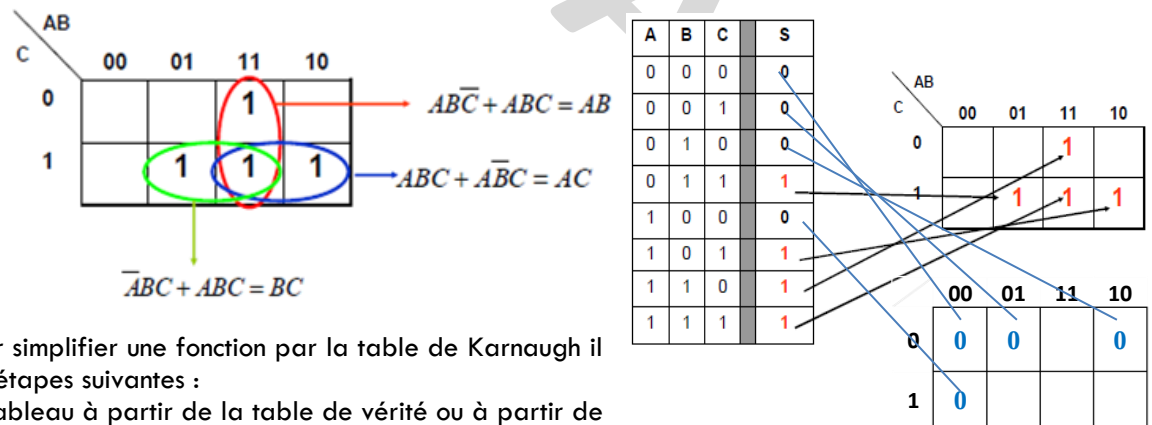
- ✓ Ces termes sont **dits adjacents**.
- ✓ Ainsi, la méthode de Karnaugh se base sur la **règle précédente (l'adjacence)**. La méthode consiste à mettre en évidence par une méthode **graphique** (un tableau) tous les termes qui sont adjacents (qui ne diffèrent que par l'état d'une seule variable). Elle peut s'appliquer aux fonctions logiques de **2, 3, 4, 5 et 6 variables**.
- ✓ Un tableau de Karnaugh comporte **2^n cases** (N est le nombre de variables).

Dans un tableau de karnaugh, chaque case possède un certain nombre de **cases adjacentes**.



5.2.2. PRINCIPE DE SIMPLIFICATION

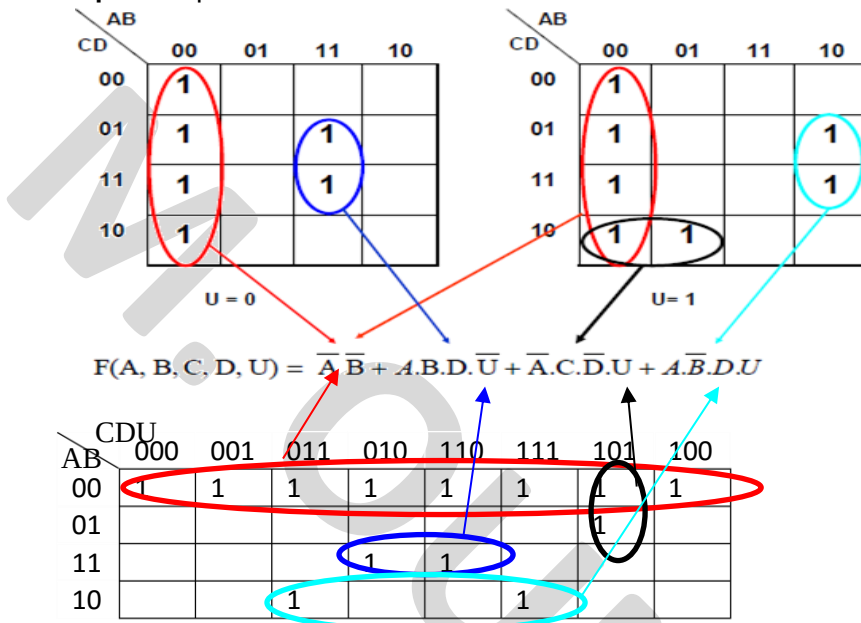
- ✓ Si la fonction logique est donnée sous la **première forme canonique** (disjonctive), alors sa représentation est directe : pour **chaque terme** lui correspond **une seule case qui doit être mise à 1**.
- ✓ Si la fonction logique est donnée sous la **deuxième forme canonique** (conjonctive), alors sa représentation est directe : pour chaque terme lui correspond **une seule case qui doit être mise à 0**.



En résumé pour simplifier une fonction par la table de Karnaugh il faut suivre les étapes suivantes :

1. **Remplir** le tableau à partir de la table de vérité ou à partir de la forme canonique.
2. Faire des **regroupements** des **cases adjacentes** qui comportent des **1**: des regroupements de 16, 8, 4, 2, 1 cases (Les **même termes** peuvent participer à plusieurs regroupements), et On s'arrête lorsqu'il y a plus de **1** en dehors des regroupements
3. Dans un regroupement qui contient :
 - 1 seul terme on ne peut pas éliminer de variables.
 - 2 termes on peut éliminer une variable (celle qui change d'état).
 - 4 termes on peut éliminer 2 variables.
 - 8 termes on peut éliminer 3 variables.
 - 16 termes on peut éliminer 4 variables.
4. L'expression **logique finale** est la réunion (la somme) des groupements après simplification et élimination des variables qui changent d'état.

Exemple : Simplification d'une fonction avec 5 variables.



La simplification des fonctions logiques en utilisant les tableaux de Karnaugh est simple, efficace et conduit à une solution minimale, mais elle devient inutilisable lorsque les dimensions de la table sont trop importantes (au-delà de 6 variables d'entrées). Dans ce cas, il faut utiliser des méthodes plus élaborées, comme celle de Quine Mc Cluskey, Programmable sur un certain nombre de logiciels spécialisés (Espresso, Mc Boole), elle est très efficace et peut proposer toutes les solutions minimales existantes.

5.3. MÉTHODE DE QUINE-MAC CLUSKEY

Bien que peu employée, nous devons quand même citer une méthode différente de celle des tableaux de Karnaugh pour la simplification des fonctions de plusieurs variables. Il s'agit d'une méthode imaginée par le mathématicien américain **W. V. QUINE** et remodelée par son compatriote le **Docteur Edwards J. Mac CLUSKEY Junior** dans une thèse de doctorat qu'il présenta au **M. I. T.** (Massachusetts Institute of Technology) en juin 1956.

5.3.1. DESCRIPTION DE LA MÉTHODE

Pour examiner cette méthode, nous prendrons un exemple à quatre variables. Partons de la table de vérité de la fonction $f(a, b, c, d)$ qui est donnée à la figure 54. Nous pouvons constater que la fonction f est "vraie" (égale à 1) pour onze combinaisons des variables a, b, c et d .

Donnons comme nom à chacune de ces combinaisons la valeur décimale qui correspond au code binaire formé par les variables a, b, c, d en considérant que a est le bit de poids fort et d le bit de poids faible.

La méthode de Quine-Mc Cluskey est utilisée lorsque la fonction est représentée par la première forme canonique (ou deuxième forme canonique) ou chaque intersection contient toutes les variables de la fonction.

Elle consiste tout d'abord à classer les intersections canoniques monômes selon leurs poids.

- ✓ On appelle **poids** d'un monôme complet, le poids de son équivalent binaire.

a	b	c	d	f
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Fig. 54. - Table de vérité.

- ✓ On appelle l'**équivalent binaire** le nombre binaire obtenu en remplaçant toutes les variables en négation (variables barrées) par 0 et autres (variables non barrées) par des 1.
- ✓ On appelle poids d'un nombre binaire le nombre de 1 contenu dans ce nombre où le nombre de lettre non barrée dans ce monôme.
- Exemple : monôme complet : $ab\bar{c}\bar{d}$
 - Équivalent binaire 1100
 - Poids : 2

Remarque : le poids maximum = nombre de variable de la fonction.

Une fois que le classement fait, on compare chaque terme de chaque groupe avec chaque terme de groupe de poids immédiatement supérieur. En utilisant l'identité suivante : $Ax + A\bar{x} = A$

A : représente l'ensemble des variables qui ne changent pas d'état.

x : la variable qui change état. On obtiendra ainsi une 1ère forme réduite. Que l'on réduira de nouveau jusqu'à que ne soit plus possible de faire des réductions.

Exemple :

$ab\bar{c}\bar{d}1100$ nous l'appellerons la combinaison 12 puisque 1100 en binaire équivaut à 12 en décimal. Si nous récapitulons, la fonction f est donc la somme des combinaisons résumées figure 55. Cette liste peut être récapitulée de la façon suivante :

$$f = \sum (0, 1, 2, 4, 6, 8, 9, 12, 13, 14, 15)$$

Fonction
«somme de»
Liste des combinaisons

A partir de cette liste de combinaisons commence réellement la méthode de QUINE-MAC CLUSKEY.

La première chose à faire est de classer ces combinaisons en ensembles successifs en fonction du nombre de zéros (ou nombre de 1) de la forme binaire de chaque combinaison en commençant par celle qui en compte le plus (qui compte le moins).

a	b	c	d	Combinaisons
0	0	0	0	Combinaison 0
0	0	0	1	Combinaison 1
0	0	1	0	Combinaison 2
0	1	0	0	Combinaison 4
0	1	1	0	Combinaison 6
1	0	0	0	Combinaison 8
1	0	0	1	Combinaison 9
1	1	0	0	Combinaison 12
1	1	0	1	Combinaison 13
1	1	1	0	Combinaison 14
1	1	1	1	Combinaison 15

Fig. 55. - Combinaisons pour lesquelles $f = 1$.

Le premier ensemble est donc formé par la combinaison 0 dont la forme binaire 0000 comporte quatre zéros. Le second ensemble regroupe les combinaisons comportant trois zéros (un seul 1) ... jusqu'au cinquième qui est formé de la combinaison 15 dont la forme binaire 1111 ne comporte pas de zéro du tout (tous des uns). Les cinq ensembles sont représentés figure 56.

La seconde phase de la méthode consiste à comparer les termes de chaque ensemble avec les termes de l'ensemble qui suit immédiatement de façon à éliminer une variable si cela est possible.

La combinaison 0000 de l'ensemble 1 doit donc être comparée aux quatre combinaisons de l'ensemble 2. Puis chacune des quatre combinaisons de cet ensemble 2 sera comparée à chacune des trois combinaisons de l'ensemble 3 et ainsi de suite jusqu'à l'ensemble 5...

	a	b	c	d	
Combinaison 0	0	0	0	0	ensemble 1 (4 zéros)
Combinaison 1	0	0	0	1	ensemble 2 (3 zéros)
Combinaison 2	0	0	1	0	
Combinaison 4	0	1	0	0	
Combinaison 8	1	0	0	0	
Combinaison 6	0	1	1	0	ensemble 3 (2 zéros)
Combinaison 9	1	0	0	1	
Combinaison 12	1	1	0	0	
Combinaison 13	1	1	0	1	ensemble 4 (1 zéro)
Combinaison 14	1	1	1	0	
Combinaison 15	1	1	1	1	ensemble 5 (0 zéro)

Fig. 56. - Classement des combinaisons en ensembles.

Lorsque deux combinaisons ne diffèrent que d'une variable, celles-ci peuvent être associées pour n'en former plus qu'une dans laquelle la variable qui diffère peut être éliminée (l'adjacence). Quand une variable s'élimine dans une telle association, on signale ce fait en remplaçant cette variable par une croix (ou tout autre signe à votre convenance).

Dans l'exemple que nous avons choisi, les combinaisons 0 et 1 peuvent, par exemple, être associées puisque seule la variable **d** est différente.

Combinaison 0 : 0 0 0 0

Combinaison 1 : 0 0 0 1

Combinaison 0 - 1: 0 0 0 X

Procédons ainsi en comparant chaque terme de chaque ensemble avec chaque terme de l'ensemble suivant et nous obtenons les combinaisons décrites dans la figure 57.

Nous obtenons de nouveaux membres regroupés à présent dans quatre ensembles I, II, III et IV. Recommençons la même opération avec ces nouveaux ensembles. La figure 58 donne les nouvelles combinaisons obtenues.

	a	b	c	d	
0 - 1	0	0	0	X	ensembles 1 et 2 I
0 - 2	0	0	X	0	
0 - 4	0	X	0	0	
0 - 8	X	0	0	0	
1 - 9	X	0	0	1	ensembles 2 et 3 II
2 - 6	0	X	1	0	
4 - 6	0	1	X	0	
4 - 12	X	1	0	0	
8 - 9	1	0	0	X	
8 - 12	1	X	0	0	
6 - 14	X	1	1	0	ensembles 3 et 4 III
9 - 13	1	X	0	1	
12 - 13	1	1	0	X	
12 - 14	1	1	X	0	
13 - 15	1	1	X	1	ensembles 4 et 5 IV
14 - 15	1	1	1	X	

Fig. 57. - Combinaisons des ensembles 1, 2, 3, 4 et 5.

			a	b	c	d
ensembles I et II	0 - 1	8 - 9	X	0	0	X
	0 - 2	4 - 6	0	X	X	0
	0 - 4	2 - 6	0	X	X	0
	0 - 4	8 - 12	X	X	0	0
	0 - 8	1 - 9	X	0	0	X
	0 - 8	4 - 12	X	X	0	0
ensembles II et III	4 - 6	12 - 14	X	1	X	0
	4 - 12	6 - 14	X	1	X	0
	8 - 9	12 - 13	1	X	0	X
	8 - 12	9 - 13	1	X	0	X
ensembles III et IV	12 - 13	14 - 15	1	1	X	X
	12 - 14	13 - 15	1	1	X	X

Fig. 58. - Combinaisons des ensembles I, II, III, IV.

Dans les principes généraux de la méthode de QUINE-MAC CLUSKEY, il convient de continuer ce processus jusqu'à ce qu'il n'y ait plus de combinaison possible entre un de ces ensembles et le suivant.

Dans l'exemple choisi, nous en sommes arrivés à ce point. De plus, nous constatons dans cette figure que plusieurs combinaisons sont identiques. Nous ne garderons bien sûr que les groupements différents que nous appellerons **les impliquants premiers** : **A, B, C, D, E et F** (figure 58). La fonction **f** prend alors la

37 | Page

5.4. CONCLUSION

A la lumière de cet exemple, résolu d'une part avec la méthode de QUINE MAC CLUSKEY, d'autre part grâce à un tableau de Karnaugh, il apparaît que la seconde est nettement plus simple, rapide et séduisante que la première. Ceci explique que cette première méthode soit quasiment inemployée. Nous ne nous étendons pas davantage sur elle.

Précisons toutefois à sa décharge que cette méthode de QUINE MAC CLUSKEY s'applique quel que soit le nombre de variables alors que la méthode des tableaux de Karnaugh se complique notablement lorsque le nombre de variables augmentent et devient supérieur à 6 ou 7.

6. ETUDE ET ÉTAPES DE CONCEPTION ET DE RÉALISATION D'UN CIRCUIT NUMÉRIQUE D'UNE FONCTION LOGIQUE

Pour faire l'étude et la réalisation d'un circuit il faut suivre les étapes suivantes :

- ✓ Il faut bien comprendre le fonctionnement du système.
- ✓ Il faut définir les variables d'entrée.
- ✓ Il faut définir les variables de sortie.
- ✓ Établir la table de vérité.
- ✓ Écrire les équations algébriques des sorties (à partir de la table de vérité).
- ✓ Effectuer des simplifications (algébrique ou par tableaux de Karnaugh.....).
- ✓ Faire le schéma avec un minimum de portes logiques.

7. CAS D'UNE FONCTION NON TOTALEMENT DÉFINIE (cas facultatifs)

Il arrive parfois qu'une fonction logique ne soit pas définie pour l'ensemble des combinaisons possibles de ses variables d'entrée. Ces combinaisons, pour lesquelles les sorties sont dites **facultatives**, **impossibles** ou **interdites**, résultent souvent de la pratique où nombre de cas ne sont pas à considérer. Ce qu'on entend par le fait de dire que la sortie est facultative, c'est qu'elle peut prendre indifféremment la valeur 0 ou 1 sans affecter le comportement attendu du système. Les techniques d'optimisation profitent alors de cette tolérance pour optimiser davantage le résultat. Grâce à leur qualité d'illustration graphique, les tables de Karnaugh permettent une exploitation élégante de ces conditions favorables.

Examinons l'exemple suivant :

Une serrure de sécurité s'ouvre en fonction de quatre clés A, B, C et D. Le fonctionnement de la serrure est défini comme suit :

- $S(A,B,C,D) = 1$ si au moins deux clés sont utilisées
- $S(A,B,C,D) = 0$ sinon.
- Les clés A et D ne peuvent pas être utilisées en même temps.
- On remarque que si la clé A et D sont utilisées en même temps l'état du système n'est pas déterminé.
- Ces cas sont appelés cas **impossibles**, **interdits** ou **facultatifs**
⇒ comment représenter ces cas dans la table de vérité ?
 - Pour les cas impossibles, interdites ou facultatifs il faut mettre un **X** dans la Table de vérité et le tableau de Karnaugh.
- Il est possible d'utiliser les **X** dans des regroupements :
 - Soit les prendre comme étant des 1 si on travaille avec les **minterms**.
 - Ou les prendre comme étant des 0 si on travaille avec les **Maxterms**.
- Il ne faut pas former des regroupements qui contiennent uniquement des **X**.
- $S(A, B, C, D) = \sum(3, 5, 6, 7, 10, 12, 14) + \Phi(9, 11, 13, 15)$

A	B	C	D	S
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	X
1	0	1	0	1
1	0	1	1	X
1	1	0	0	1
1	1	0	1	X
1	1	1	0	1
1	1	1	1	X

- $S(A, B, C, D) = \prod (0, 1, 2, 4, 8) + \Phi(9, 11, 13, 15)$

